KIROVAULT

SQLite Database Schema & Search

Local index and metadata design for incremental discovery, virtual collections and responsive search.

DOCUMENT PURPOSE

This standalone companion document complements the KiroVault Desktop App Blueprint. It follows the Windows-first, local-first product direction. Recommendations are proposed guidance, not claims that these features already exist.

Contents and implementation guidance

Core tables

• files(id, canonical_path, parent_path, name, extension, kind, size_bytes, created_at, modified_at, indexed_at, source_root, scan_status).

• media_metadata(file_id, width, height, duration_ms, codec/container hints, capture_time, audio metadata, metadata_source).

• thumbnail_cache(cache_key, file/path key, thumbnail reference, dimensions, generator_version, source_fingerprint, created_at).

• collections(id, name, type, query_json/rule, created_at); favorites(id, path, item_kind, pinned_at).

• tags(id, name, color); file_tags(file_id/path key, tag_id); settings(key, value_json, updated_at).

• scan_jobs(id, root_path, status, cursor/checkpoint, counters, errors, timestamps); optional content_fts.

Schema rules

• Use migrations and schema versioning; keep the index rebuildable.

• Use prepared statements and parameter binding for every user-controlled value.

• Persist canonical paths and revalidate before opening a result; external moves/deletes make records stale.

• Separate original files from thumbnails and index data; database deletion must never delete originals.

• Consider SQLite WAL only after reviewing workload and official caveats; avoid shared network-mounted WAL databases.

Search tiers

• Immediate folder search: name, extension and current folder scope, available before full scan.

• Indexed local search: name, path, type, size, dates and metadata.

• Advanced filters: location, type, size/date ranges, favorites, tags and duplicate candidates.

• Optional content search: extracted text from supported files only, with explicit opt-in and scope.

Index lifecycle

• Allow users to select roots and exclude paths/extensions/size classes.

• Support incremental scan, pause/resume/cancel/rebuild and last-scan/error summaries.

• Mark removable-drive data stale on disconnect; revalidate on reconnect.

• Use indexes for parent path, name, extension, modification time and other measured query needs.

Validation

• Test migration from each supported schema version and recovery from interrupted writes.

• Test duplicate/stale path reconciliation and Unicode filenames.

• Measure query latency with large synthetic indexes.

• Verify FTS content indexing is disabled unless explicitly enabled.

References

- SQLite documentation: https://www.sqlite.org/docs.html
- SQLite FTS5: https://www.sqlite.org/fts5.html
- SQLite WAL: https://www.sqlite.org/wal.html
- KiroVault Desktop App Blueprint, p. 18 (user-provided source).

Reference note: external documentation may change. Verify current versions, licensing, APIs and platform prerequisites during implementation.